

Kubernetes Deployment Assignment

Submitted By:
Dhinakaran J

Batch 03 - Assignment 3 - Kubernetes

Introduction

This assignment demonstrates deployment and management of an application using Kubernetes. The project includes containerization using Docker, deployment using Kubernetes resources, rolling updates, and health monitoring using probes.

Objective

The main objective of this assignment is to understand:

- Pod creation
- ReplicaSet management
- Deployment management
- Rolling updates
- Kubernetes Services
- Container health checks using probes

Frontend Application

The application is a simple frontend UI created using HTML and CSS.

Version 1

Displays:

Version 1
Done by JD

Version 1

Done by JD

Version 2

Displays:

Version 2
Done by JD

Version 2

Done by JD

Docker Setup

Dockerfile

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
```

Docker Build Commands

Build v1

```
docker build -t dhinakaran2kjd/k8s-app:v1 .
```

```
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V1> docker build -t dhinakaran2kjd/k8s-app:v1 .
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4d4de5878cf7edcb475e59193904b198d9b830de
=> => resolve docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4d4de5878cf7edcb475e59193904b198d9b830de
=> => sha256:453da7dbc73e7338c4e201ec3f3c4a5f7751adbf5a6f127792c1cd001a780721 1.21kB / 1.21kB
=> => sha256:612c0c1df4c55a0bf145f84df03cb28de505e6a52fb8e49c9da7b143fc00ad2d 19.92MB / 20.25MB
=> => sha256:4a8b0b2a5b1937755263ac7ac4ee26db2906c13a5d70f1fb1875ba8e370cdd8f 1.40kB / 1.40kB
=> => sha256:781ff50d2644f74714c0d5a4ffa3447bbfd4b293cfb1dc6a637722ae24c89241 404B / 404B
=> => sha256:aee4e54b3865ee4d98545b6b49f9e8ab3b9b6ed114a2eafedc7a22e984ea5a68 956B / 956B
=> => sha256:583599bb7d382e9e986a9ff65204981307bfa71670a29f4190eedae93c4858be 627B / 627B
=> => sha256:82736a35d0e7f1309edc13d09115410b81542cf8b91afff80deeace02b6f2f26 1.87MB / 1.87MB
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
=> => extracting sha256:82736a35d0e7f1309edc13d09115410b81542cf8b91afff80deeace02b6f2f26
=> => extracting sha256:583599bb7d382e9e986a9ff65204981307bfa71670a29f4190eedae93c4858be
=> => extracting sha256:aee4e54b3865ee4d98545b6b49f9e8ab3b9b6ed114a2eafedc7a22e984ea5a68
=> => extracting sha256:781ff50d2644f74714c0d5a4ffa3447bbfd4b293cfb1dc6a637722ae24c89241
=> => extracting sha256:453da7dbc73e7338c4e201ec3f3c4a5f7751adbf5a6f127792c1cd001a780721
=> => extracting sha256:4a8b0b2a5b1937755263ac7ac4ee26db2906c13a5d70f1fb1875ba8e370cdd8f
=> => extracting sha256:612c0c1df4c55a0bf145f84df03cb28de505e6a52fb8e49c9da7b143fc00ad2d
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:71eda288ff6f773efb4dc1af8680afb13d54511b7b79654e62e0064620aa5ed6
=> => exporting config sha256:d872cd005145c5a614dd4e90e80ec47ed615109904d015b1b73d8eaa65e35dbd
=> => exporting attestation manifest sha256:f6a43f9a66f8651693132d25bfae6cc274e93e5124cef28de4c95b786f665e33
=> => exporting manifest list sha256:358bfc1d7d44c4d2cfe8e581464271ca0015274ee271f8de61a3b9273388a135
=> => naming to docker.io/dhinakaran2kjd/k8s-app:v1
=> => unpacking to docker.io/dhinakaran2kjd/k8s-app:v1
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V1> 
```

Build v2

```
docker build -t dhinakaran2kjd/k8s-app:v2 .
```

```

PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2> docker build -t dhinakaran2kjd/k8s-app:v2 .
[+] Building 15.8s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 119B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
=> => resolve docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
=> [internal] load build context
=> => transferring context: 282B
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:70ffbdcd80ee611b56a193468e98bbc402fcb58fec7a710787951eb5b31d246e
=> => exporting config sha256:ea1413296948633ffff3828e8ee563279f5f40325b02ce137eff42908081e5f3
=> => exporting attestation manifest sha256:ea72fc470efa171b3447c80c4cb5bb21233ee5c6689b65198997496715ec865f
=> => exporting manifest list sha256:9dba6f3f0b30a7a92d290fb5ae3a54df19700d475db92e92f4569f83f4e04f43
=> => naming to docker.io/dhinakaran2kjd/k8s-app:v2
=> => unpacking to docker.io/dhinakaran2kjd/k8s-app:v2
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2>

```

Docker Push Commands

docker push dhinakaran2kjd/k8s-app:v1

```

PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2> docker push dhinakaran2kjd/k8s-app:v1
The push refers to repository [docker.io/dhinakaran2kjd/k8s-app]
a992bbd33a86: Pushed
82736a35d0e7: Pushed
aee4e54b3865: Pushed
781ff50d2644: Pushed
453da7dbc73e: Pushed
6a0ac1617861: Pushed
583599bb7d38: Pushed
1a8a4f43441d: Pushed
4a8b0b2a5b19: Pushed
612c0c1df4c5: Pushed
v1: digest: sha256:358bfc1d7d44c4d2cfe8e581464271ca0015274ee271f8de61a3b9273388a135 size: 856
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2>

```

docker push dhinakaran2kjd/k8s-app:v2

```

PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2> docker push dhinakaran2kjd/k8s-app:v2
The push refers to repository [docker.io/dhinakaran2kjd/k8s-app]
34d3a899fab1: Pushed
583599bb7d38: Layer already exists
aee4e54b3865: Layer already exists
453da7dbc73e: Layer already exists
4a8b0b2a5b19: Layer already exists
612c0c1df4c5: Layer already exists
80d2e5c1ecd5: Pushed
82736a35d0e7: Layer already exists
781ff50d2644: Layer already exists
6a0ac1617861: Layer already exists
v2: digest: sha256:9dba6f3f0b30a7a92d290fb5ae3a54df19700d475db92e92f4569f83f4e04f43 size: 856
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\V2>

```

Kubernetes Files

pod. yaml

- Creates a single pod

- Runs container using v1 image

```
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl apply -f .\pod.yaml
pod/frontend-pod created
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
frontend-pod  0/1     ContainerCreating  0          11s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
frontend-pod  0/1     ContainerCreating  0          17s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods -w
NAME          READY   STATUS             RESTARTS   AGE
frontend-pod  1/1     Running            0          27s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
frontend-pod  1/1     Running            0          32s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s>
```

ReplicaSet (rs. yaml)

- Maintains multiple pod replicas
- Ensures high availability

```
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl apply -f .\rs.yaml
replicaset.apps/frontend-rs created
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
apiagggregator-agent-59cbf6f854     0         0         0       18d
echo-agent-8555bc758b               0         0         0       19d
frontend-rs                         3         3         3       12s
get-crypto-price-85f997c95b         0         0         0       18d
sum-and-greet-fdd6cc6f4             0         0         0       19d
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s>
```

```
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
frontend-pod  1/1     Running            0          3m16s
frontend-rs-44z5p  1/1     Running            0          61s
frontend-rs-bf4bn  1/1     Running            0          61s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s>
```

deployment. yaml

- Manages application deployment
- Supports rolling updates
- Includes probes

```

PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl apply -f .\deployment.yaml
deployment.apps/frontend-deployment created
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get deployments
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
apiagggregator-agent              0/0      0              0             18d
echo-agent                        0/0      0              0             19d
frontend-deployment               0/3      3              0             7s
get-crypto-price                  0/0      0              0             18d
sum-and-greet                     0/0      0              0             19d
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
frontend-deployment-749865ff97-2mkzk 1/1      Running   0            15s
frontend-deployment-749865ff97-724dh 1/1      Running   0            15s
frontend-deployment-749865ff97-zl8dm 1/1      Running   0            15s
frontend-pod                         1/1      Running   0            5m8s
frontend-rs-44z5p                   1/1      Running   0            2m53s
frontend-rs-bf4bn                   1/1      Running   0            2m53s
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s>

```

service.yml

- Exposes application externally
- Routes traffic to pods

```

PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl apply -f .\service.yaml
service/frontend-service created
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get svc
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
apiagggregator-agent              ClusterIP   10.96.117.215  <none>          8080/TCP          18d
echo-agent                        ClusterIP   10.96.109.84   <none>          8080/TCP          19d
frontend-service                  NodePort    10.96.58.148   <none>          80:30080/TCP      9s
get-crypto-price                  ClusterIP   10.96.193.161  <none>          8080/TCP          18d
kubernetes                        ClusterIP   10.96.0.1      <none>          443/TCP           20d
sum-and-greet                     ClusterIP   10.96.12.55    <none>          3000/TCP          19d
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s>

```

```

C:\Users\npci>kubectl port-forward svc/frontend-service 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080

```

V1 Running

Browser showing:

Version 1
Done by JD



Update Deployment to v2

Run image update command:

```
kubectl set image deployment/frontend-deployment  
frontend=dhinakaran2kjd/k8s-app:v2
```

This command updates deployment image from:

- v1 → v2

Monitor Rolling Update

Watch update process:

```
kubectl get pods -w
```

During update:

- Old Pods terminate gradually
- New Pods create gradually

Example process:

Old Pod → Terminating

New Pod → Creating

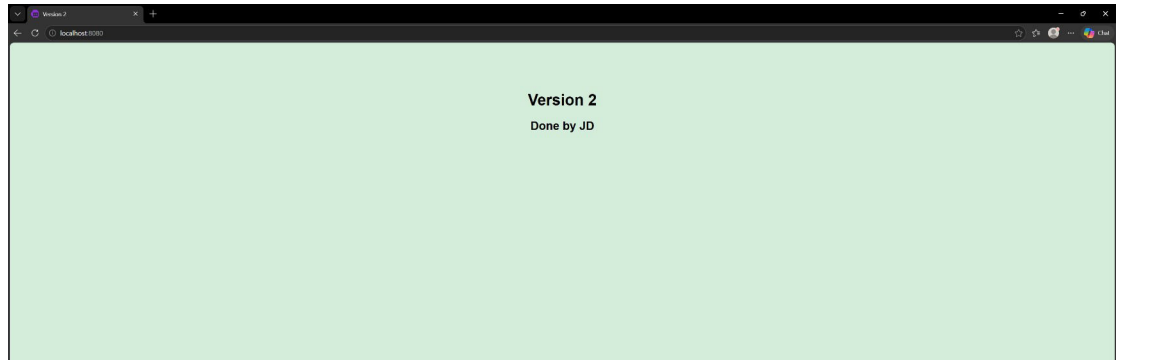
New Pod → Running

```
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl set image deployment/frontend-deployment frontend=dhinakaran2kjd/k8s-app:v2  
deployment.apps/frontend-deployment image updated  
PS C:\Users\npci\Desktop\Kubernetes-assignment\Front-end\k8s> kubectl get pods -w  
NAME                                READY   STATUS    RESTARTS   AGE  
frontend-deployment-6bdcc6d955-59jvx 1/1     Running   0           41s  
frontend-deployment-6bdcc6d955-mmsrx 1/1     Running   0           41s  
frontend-deployment-6bdcc6d955-pj126 1/1     Running   0           31s
```

v2 Running

Browser showing:

Version 2
Done by JD



Conclusion

This assignment successfully demonstrated the deployment and management of an application using Kubernetes.

The frontend application was containerized using Docker and deployed into the Kubernetes cluster using Pods, ReplicaSets, Deployments, and Services. Health monitoring was implemented using Liveness Probe, Readiness Probe, and Startup Probe.

The application was initially deployed with Version 1 and later updated to Version 2 using the Rolling Update strategy. Kubernetes successfully replaced old Pods with new Pods without downtime, ensuring continuous application availability.

This assignment provided practical understanding of:

- Docker image creation and versioning
- Kubernetes resource management
- Service exposure
- Rolling updates
- Container health monitoring
- Deployment lifecycle management

Overall, the deployment and update process was completed successfully and the application was accessible through the Kubernetes Service.